Deploying a Java Web Application Using Maven, Nexus, and Tomcat in a Multi-Server Environment

Introduction

In this project, I implemented a complete multi-server Java application deployment pipeline using:

- GitHub as Source Code Repository
- Apache Maven for Build Automation
- Nexus Repository Manager for Artifact Storage
- Apache Tomcat for Application Deployment

The goal of this Proof of Concept (POC) was to simulate a real-world DevOps deployment workflow where applications are built, stored, and deployed across separate servers.

Prerequisites

Before starting, ensure the following:

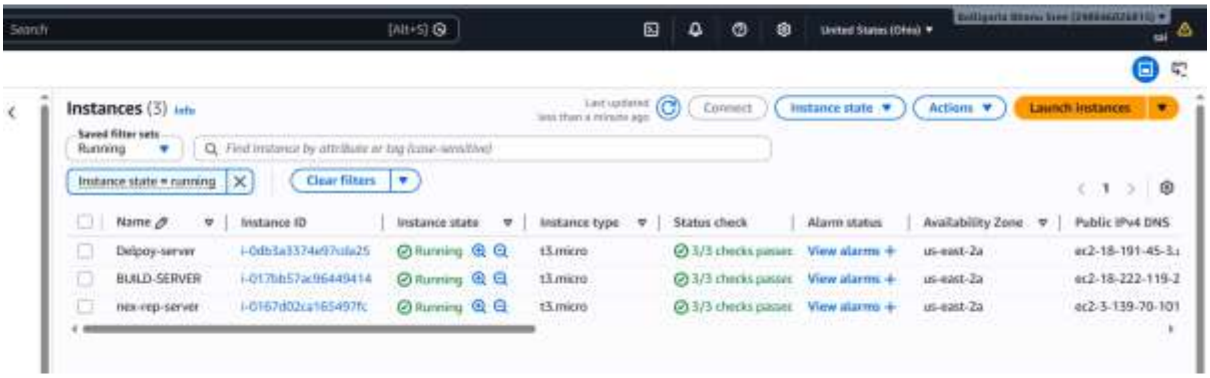
- 3 Ubuntu Servers
- Internet Connectivity
- Open Ports:
    - 22 → SSH
    - 8081 → Nexus
    - 8080 → Tomcat
- Java Installed on All Servers

Step 1 — Configure Build Server

Objective

The Build Server is responsible for:

- Cloning source code
- Building the application
- Uploading the WAR artifact to Nexus



1.2 Install Java

sudo apt install openjdk-17-jdk -y

```
See 'git help git' for an overview of the system.
ubuntu@BUILD-SERVER:~$ sudo sudo apt install openjdk-17-jdk
Installing:
  openjdk-17-jdk

Installing dependencies:
  adwaita-icon-theme          libavahi-common3          libjpeg-turbo8          libxdamage1
  alsa-topology-conf          libc-dev-bin              libjpeg8                libxdmcp-dev
```

1.3 Install Maven

sudo apt install maven –y

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@BUILD-SERVER:~$ mvn
Command 'mvn' not found, but can be installed with:
sudo apt install maven
ubuntu@BUILD-SERVER:~$ sudo apt install maven
Installing:
  maven
```

1.4 Install Git

sudo apt install git –y

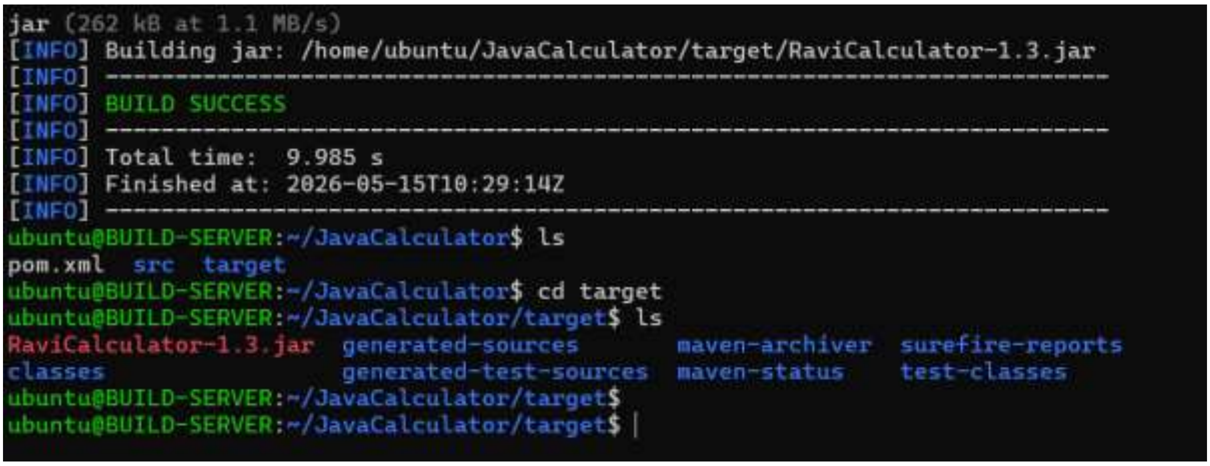
Clone GitHub Repository

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@BUILD-SERVER:~$ mvn
Command 'mvn' not found, but can be installed with:
sudo apt install maven
ubuntu@BUILD-SERVER:~$ sudo apt install maven
Installing:
  maven
```

1.5 Build the Application

Run Maven Build

mvn clean install



Step 2 — Configure Nexus Repository Server

Objective

The Nexus Server stores application artifacts generated by the Build Server.

2.1 Install Java

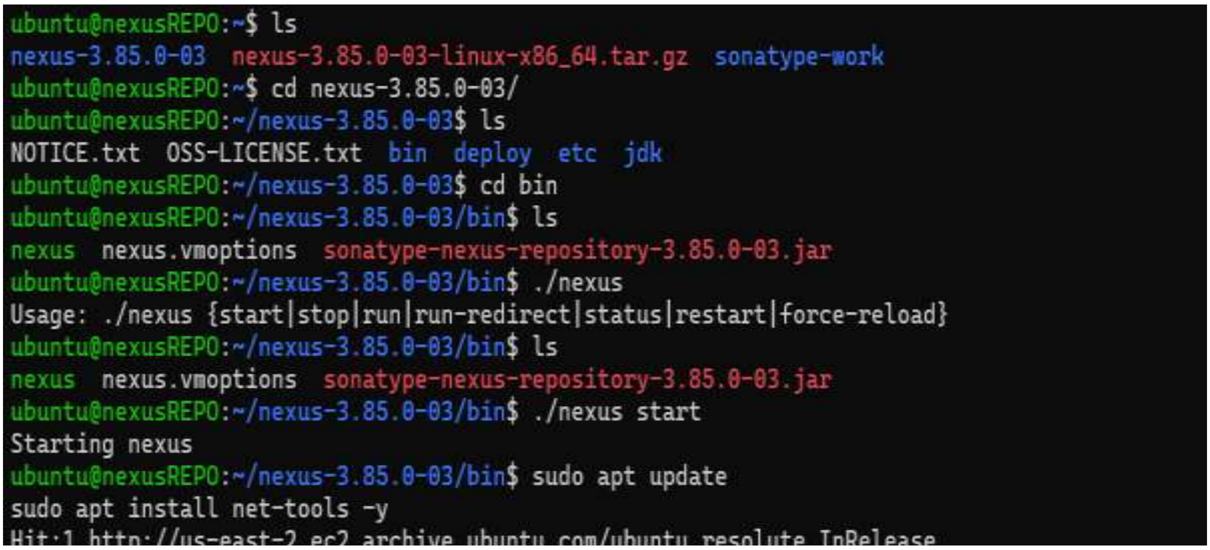
sudo apt update -y
sudo apt install openjdk-17-jdk -y

2.2 Download Nexus

Download: wget https://download.sonatype.com/nexus/3/latest-unix.tar.gz
Extract: tar -xvf latest-unix.tar.gz

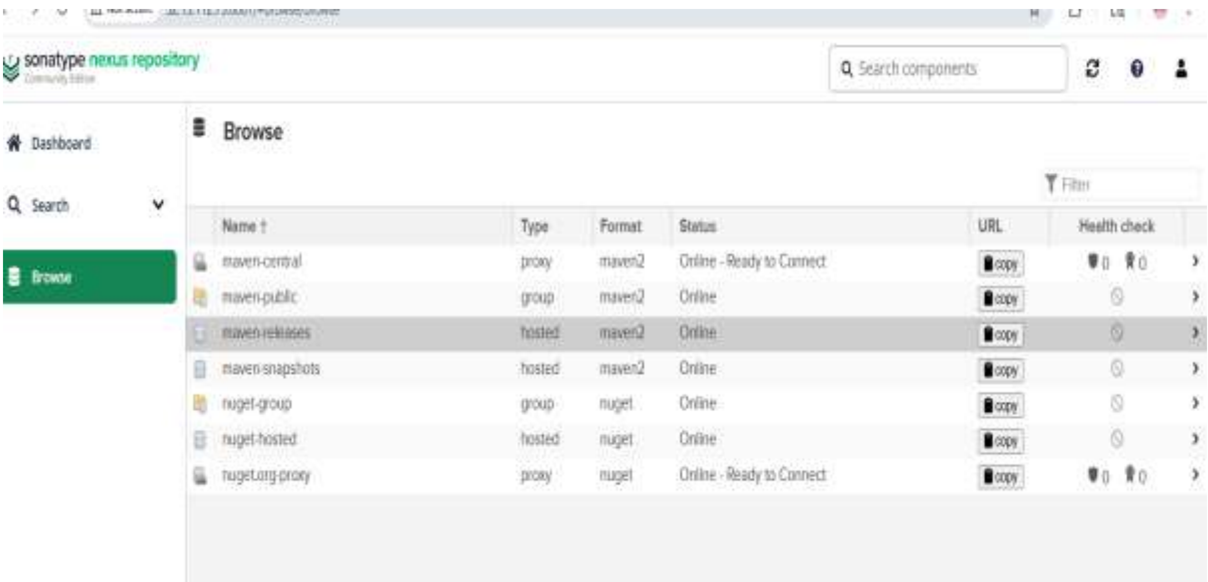
2.3 Create Nexus User

sudo adduser nexus



2.4 Access Nexus UI

Browser: http://<NEXUS_SERVER_IP>:8081



Step 3 — Configure Maven to Upload Artifact to Nexus

Objective

Configure Maven so that WAR files are automatically uploaded to Nexus.

3.1 Update pom.xml

```
                  <manifest>
                     <!-- give full qualified name of your main class-->
                     <mainClass>com.ravi.cal.RaviCalculator.Calculator</mainClass>
                  </manifest>
               </archive>
            </configuration>
         </plugin>
      </plugins>
   </build>
   <distributionManagement>
    <repository>
      <id>maven-releases</id>
      <url>http://52.15.112.73:8081/repository/maven-releases/</url>
    </repository>
  </distributionManagement>
</project>

ubuntu@BUILD-SERVER:~/JavaCalculator$ |
```

3.2 Configure Maven settings.xml

```
      <id>deploymentRepo</id>
      <username>repouser</username>
      <password>repopwd</password>
    </server>
    -->
  <servers>
    <server>
        <id>maven-releases</id>
        <username>admin</username>
        <password>admin</password>
    </server>
  </servers>

    <!-- Another sample, using keys to authenticate.
    <server>
```

3.3 Deploy Artifact to Nexus

Run: mvn clean deploy

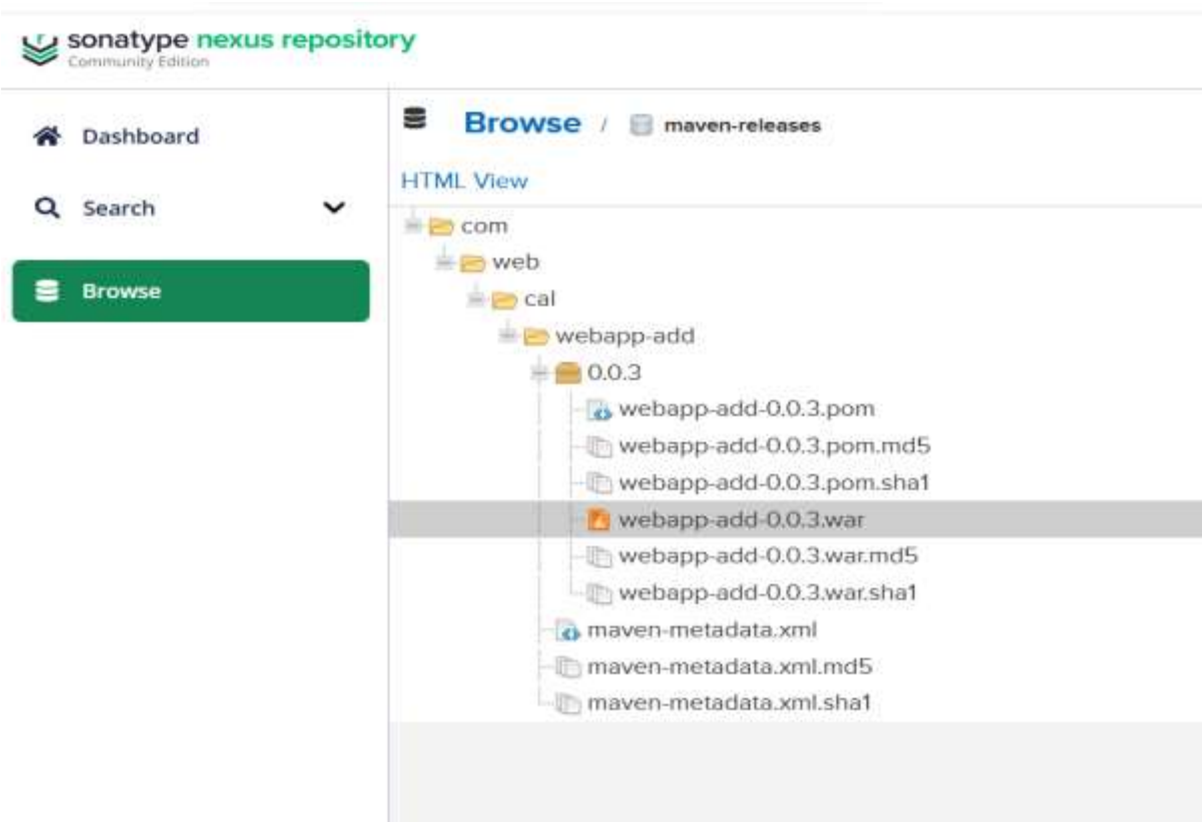
Expected Output: BUILD SUCCESS

3.4 Verify Artifact in Nexus

Navigate:

Browse → maven-releases

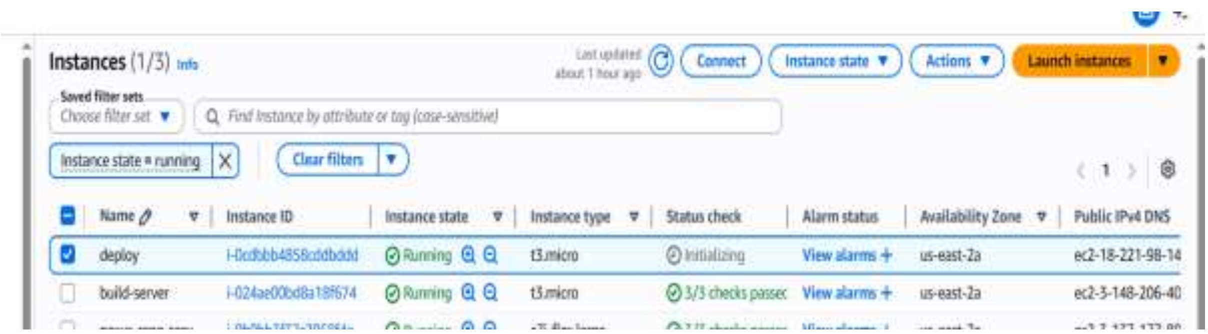
Verify WAR artifact uploaded successfully.



Step 4 — Configure Deploy Server

Objective

The Deploy Server downloads the WAR file from Nexus and deploys it into Apache Tomcat.



4.1 Install Java

```
sudo apt update -y
sudo apt install openjdk-17-jdk -y
```

```
--<name> <value>.

ubuntu@deploy:~$ java --version
openjdk 17.0.18 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-1, mixed mode, sharing)
ubuntu@deploy:~$ |
```

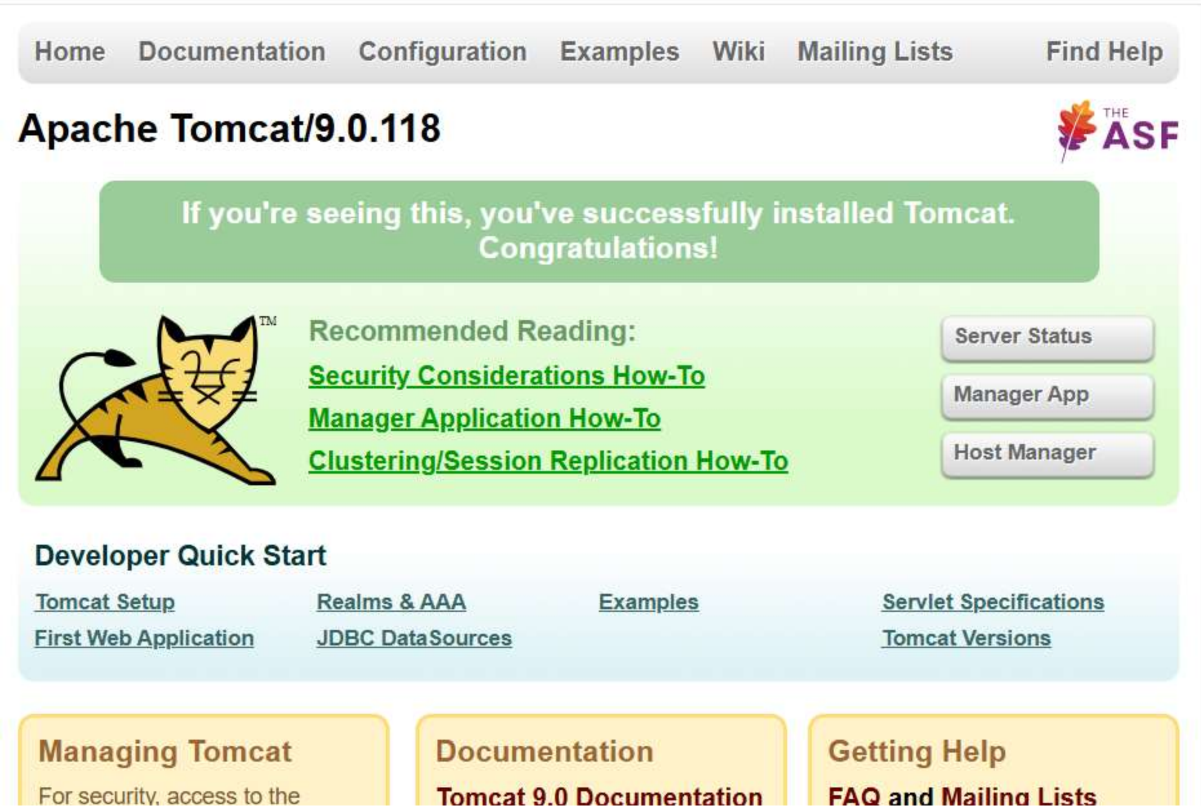
4.2 Install Tomcat

Official Website: wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.118/bin/apache-tomcat-9.0.118.tar.gz

Extract: tar -xvf apache-tomcat-9.0.118.tar.gz

4.3 Start Tomcat

cd /opt/tomcat/bin
sudo sh startup.sh



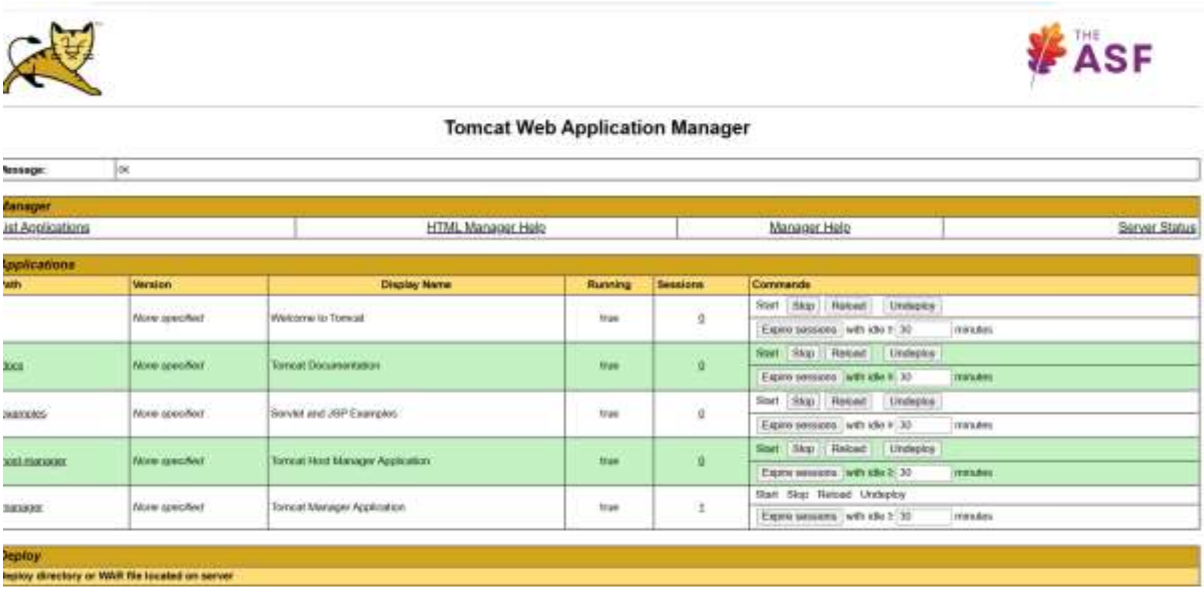
Step 5 — Configure Tomcat Manager Access

5.1 Update tomcat-users.xml

Edit: sudo vi /opt/tomcat/conf/tomcat-users.xml

5.2 Allow Remote Access

Edit: sudo vi /opt/tomcat/webapps/manager/META-INF/context.xml



Step 6 — Download WAR File from Nexus

6.1 Download Artifact

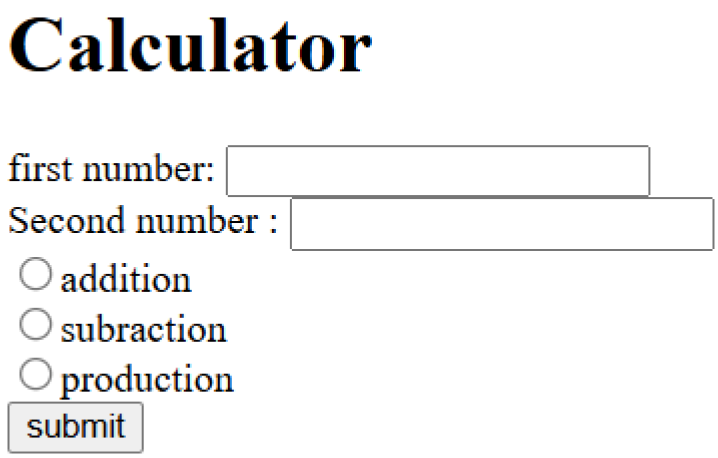
```
buntu@deploy:~/apache-tomcat-9.0.118$ cd webapps/
buntu@deploy:~/apache-tomcat-9.0.118/webapps$ ls
OOT  docs  examples  host-manager  manager
buntu@deploy:~/apache-tomcat-9.0.118/webapps$ wget http://3.137.177.89:8081/repository/maven-releases/com/web/cal/webap
-add/0.0.3/webapp-add-0.0.3.war
-2026-05-18 12:58:12--  http://3.137.177.89:8081/repository/maven-releases/com/web/cal/webapp-add/0.0.3/webapp-add-0.0.
.war
onnecting to 3.137.177.89:8081... connected.
TTP request sent, awaiting response... 200 OK
ength: 3981 (3.9K) [application/java-archive]
```

Step 7 — Deploy WAR to Tomcat

Copy WAR: cp calculator-1.0.war /opt/tomcat/webapps/

Step 8 — Access Application

Open Browser: Application will host.



Challenges Faced

Issue 1 - Tomcat Manager Access Denied

Solution

- Added roles in tomcat-users.xml
- Commented RemoteAddrValve
- Restarted Tomcat

Issue 2 - Nexus Authentication Failure

Solution

- Corrected credentials in settings.xml
- Verified repository permissions

Issue 3 - WAR Not Deploying

Solution

- Checked WAR file location
- Restarted Tomcat

- Verified Java compatibility

Conclusion

This project successfully demonstrated a real-world multi-server Java deployment pipeline using Maven, Nexus, and Apache Tomcat.